

Velepin and Marketing - omówienie

Codeforces Round 852 (Div. 2)
Problem E

Kod zadania: vel
Limit pamięci: 256 MiB



Treść

Mamy n czytelników, każdemu z nich chcemy polecić jedną z k książek. Każda książka musi być polecana przynajmniej jednej osobie. Osoba i -ta jest zadowolona, jeśli co najmniej a_i osób (włącznie z nią) czyta tę samą książkę co ona. Dostajemy q zapytań, musimy zmaksymalizować liczbę zadowolonych osób w zależności od parametru k ($1 \leq n, q \leq 300\,000$).

Obserwacje

Zacznijmy od posortowania niemalejąco ciągu a . Powiedzmy że mamy już jakiś podział na grupy i wiemy kto jest szczęśliwy a kto nie. Jeżeli istnieje jakaś para indeksów i, j taka że $a_i < a_j$, to nie opłaca nam się żeby j był zadowolony a i nie. W takiej sytuacji wystarczy zamienić ich przydziały do grup, a wynik na pewno się nie pogorszy. Stąd wniosek, że w optymalnym rozwiązaniu osoby zadowolone tworzą jakiś **prefiks** (posortowanego) ciągu a .

Brut

Skupmy się na tylko jednym zapytaniu. Spróbujmy dla każdego x stwierdzić, czy istnieje jakiś podział, w którym dokładnie x pierwszych osób jest zadowolone. Jeżeli $k < n - x$, to optymalnie jest zrobić grupę z pierwszych $n - k + 1$ osób, a do reszty grup przydzielić po jednej osobie (której zadowolenie jest nam obojętne).

W przeciwnym przypadku, podzielimy prefiks x na maksymalną liczbę grup, a resztę podzielimy znowu na 1-osobowe grupy. Głównym wyzwaniem jest tu stworzenie wystarczającej ilości grup, więc mieszanie prefiksu x z innymi osobami nie ma sensu. Dowód tego pozostawiam jako ćwiczenie dla czytelnika :).

Możemy już skonstruować pierwszy algorytm. Niech

$dp[i]$ – maksymalna liczba grup, na które możemy podzielić pierwsze i osób tak, żeby każda z nich była zadowolona
Jeżeli $a[i] > i$, to ustawiamy $dp[i] = 0$. Inaczej stworzymy grupę z co najmniej a_i ostatnich osób (na prefiksie). Zatem

$$dp[i] = \left((a_i > i) ? 0 : 1 + \max_{0 \leq j \leq i - a_i} dp[j] \right)$$

Na zapytania będziemy odpowiadać liniowo, sprawdzając każdy możliwy x . Jak ustaliliśmy wcześniej, musi zachodzić

$$a_x \leq n - k + 1 \wedge k < n - x$$

(jeżeli x -ta jest zadowolona to też wszystkie przed nią), albo

$$dp[x] + n - x \geq k \wedge k \geq n - x$$

Wzorcówka

Zacniemy od przyspieszenia naszego dp -ka. Wystarczy utrzymywać jego maksima prefiksowe, wtedy całość obliczymy w czasie liniowym. Jak zoptymalizować odpowiadanie na zapytania?

Będziemy binarnie wyszukiwać odpowiedzi na dane zapytanie. Oczywiście jeżeli da się zadowolić x osób, to tym bardziej da się zadowolić mniej niż x osób. Dany x sprawdzamy dokładnie tak jak wcześniej. Ostateczna złożoność: $O(n + q \cdot \log_2 n)$.



Alternatywne rozwiązanie

Spróbujemy w inny sposób przyspieszyć naszego bruta. Patrząc na ograniczenia, w najgorszym przypadku będziemy musieli policzyć odpowiedź dla wszystkich możliwych k . Rozwiążemy więc nasze zadanie offline, preprocessując odpowiedzi na każde zapytanie przed wczytywaniem ich. Spójrzmy jeszcze raz na warunek zadowolenia x osób przy parametrze k . Przekształcając, otrzymujemy $k \leq \min(n - x, n + 1 - a_x)$ albo $n - x < k \leq n + dp[x] - x$.

Założmy, że nasze odpowiedzi trzymamy w ciągu odp o długości $n + 1$ (od 0 do n). Wtedy wystarczy dla danego x wykonać operację $chmax$ x na przedziałach $[0, \min(n - x, n + 1 - a_x)]$ i $[n - x + 1, n + dp[x] - x]$. Użyjemy w tym celu drzewa przedziałowego typu przedział-punkt. Złożoność tak jak wcześniej.